

附录 A 计算机系统结构模拟实验

A.0 实验平台

A.0.1 平台地址

学院服务器：

Publicfiles (\\10.60.41.1) (Z:) ▶ CourseDocuments_课程文档 ▶ ComputerArchitecture_张晨曦 ▶ 实验 ▶

A.1 实验一 MIPS 指令系统和 MIPS 体系结构

A.1.1 实验目的

1. 了解和熟悉指令级模拟器。
2. 熟练掌握 MIPSsim 模拟器的操作和使用方法。
3. 熟悉 MIPS 指令系统及其特点，加深对 MIPS 指令操作语义的理解。
4. 熟悉 MIPS 体系结构。

A.1.2 实验平台

实验平台采用指令级和流水线操作级模拟器 MIPSsim。
环境的建立：见 A.0。

A.1.3 实验内容和步骤

首先要阅读 MIPSsim 模拟器的使用方法(见附录 B)，然后了解 MIPSsim 的指令系统(见附录 C)。

1. 启动 MIPSsim (用鼠标双击 MIPSsim.exe)。
2. 点击“配置”→“流水方式”，使模拟器工作在非流水方式下。
3. 阅读附录，熟悉 MIPSsim 模拟器的操作和使用方法。

可以先载入一个样例程序(在本模拟器所在的文件夹下的“样例程序”文件夹中)，然后分别以单步执行一条指令、执行多条指令、连续执行、设置断点等方式运行程序，观察程序的执行情况，观察 CPU 中寄存器和存储器的内容的变化。

4. 选择“文件”→“载入程序”选项，加载样例程序 alltest.s，然后查看“代码”窗口，查

看程序所在的位置（起始地址为 0x00000000）。

5. 查看“寄存器”窗口 PC 寄存器的值：[PC] = 0x_____。
6. 执行 load 和 store 指令。步骤如下：
 - (1) 单步执行一条指令（F7）。
 - (2) 下一条指令地址为 0x_____，是一条____（有，无）符号载入_____（字，半字，字）指令。
 - (3) 单步执行 1 条指令（F7）。
 - (4) 查看 R1 的值，[R1] = 0x_____。
 - (5) 下一条指令地址为 0x_____，是一条____（有，无）符号载入_____（字，半字，字）指令。
 - (6) 单步执行 1 条指令。
 - (7) 查看 R1 的值，[R1] = 0x_____。
 - (8) 下一条指令地址为 0x_____，是一条____（有，无）符号载入_____（字，半字，字）指令。
 - (9) 单步执行 1 条指令。
 - (10) 查看 R1 的值，[R1] = 0x_____。
 - (11) 单步执行 1 条指令。
 - (12) 下一条指令地址为 0x_____，是一条保存_____（字，半字，字）指令。
 - (13) 单步执行 1 条指令。
 - (14) 查看内存 BUFFER 处字的值，值为 0x_____。
7. 执行算术运算类指令。步骤如下：
 - (1) 双击“寄存器”窗口里的 R1，将其值修改为 2。
 - (2) 双击“寄存器”窗口里的 R2，将其值修改为 3。
 - (3) 单步执行 1 条指令。
 - (4) 下一条指令地址为 0x_____，是一条加法指令。
 - (5) 单步执行 1 条指令。
 - (6) 查看 R3 的值，[R3] = 0x_____。
 - (7) 下一条指令地址为 0x_____，是一条乘法指令。
 - (8) 单步执行 1 条指令。
 - (9) 查看 LO、HI 的值，[LO] = 0x_____，[HI] = 0x_____。
8. 执行逻辑运算类指令。步骤如下：
 - (1) 双击“寄存器”窗口里的 R1，将其值修改为 0xFFFF0000。
 - (2) 双击“寄存器”窗口里的 R2，将其值修改为 0xFF00FF00。
 - (3) 单步执行 1 条指令。
 - (4) 下一条指令地址为 0x_____，是一条逻辑与运算指令，第二个操作数寻址方式是_____（寄存器直接寻址，立即数寻址）。
 - (5) 单步执行 1 条指令。
 - (6) 查看 R3 的值，[R3] = 0x_____。
 - (7) 下一条指令地址为： 0x_____，是一条逻辑与指令，第二个操作数寻址方式是_____（寄存器直接寻址，立即数寻址）。
 - (8) 单步执行 1 条指令。
 - (9) 查看 R3 的值，[R3] = 0x_____。
9. 执行控制转移类指令。步骤如下：

- (1) 双击“寄存器”窗口里的 R1，将其值修改为 2。
- (2) 双击“寄存器”窗口里的 R2，将其值修改为 2。
- (3) 单步执行 1 条指令。
- (4) 下一条指令地址为 0x，是一条 BEQ 指令，其测试条件是： ，目标地址为 0x。
- (5) 单步执行 1 条指令。
- (6) 查看 PC 的值，[PC]=0x，表明分支 （成功，失败）。
- (7) 下一条指令地址是一条 BGEZ 指令，其测试条件是 ，目标地址为 0x。
- (8) 单步执行 1 条指令。
- (9) 查看 PC 的值，[PC]=0x，表明分支 （成功，失败）。
- (10) 下一条指令地址是一条 BGEZAL 指令，其测试条件是 ，目标地址为 0x。
- (11) 单步执行 1 条指令。
- (12) 查看 PC 的值，[PC]=0x，表明分支 （成功，失败）；
查看 R31 的值，[R31]=0x。
- (13) 单步执行 1 条指令。
- (14) 查看 R1 的值，[R1]= 0x。
- (15) 下一条指令地址为 0x，是一条 JALR 指令，保存目标地址的寄存器为 R，保存返回地址的目标寄存器为 R。
- (16) 单步执行 1 条指令。
- (17) 查看 PC 和 R3 的值，[PC]=0x，
[R3]= 0x。

A.2 实验二 流水线及流水线中的冲突

A.2.1 实验目的

1. 加深对计算机流水线基本概念的理解。
2. 理解 MIPS 结构如何用 5 段流水线来实现，理解各段的功能和基本操作。
3. 加深对数据冲突、结构冲突的理解，理解这两类冲突对 CPU 性能的影响。
4. 进一步理解解决数据冲突的方法，掌握如何应用定向技术来减少数据冲突引起的停顿。

A.2.2 实验平台

实验平台采用指令级和流水线操作级模拟器 MIPSsim。
环境的建立：见 A.0。

A.2.3 实验内容和步骤

首先要掌握 MIPSsim 模拟器的使用方法。详见附录 B。

1. 启动 MIPSsim。
2. 根据预备知识中关于流水线各段操作的描述，进一步理解流水线窗口中各段的功能，掌握各流水寄存器的含义。（用鼠标双击各段，就可以看到各流水寄存器的内容）
3. 熟悉 MIPSsim 模拟器的操作和使用方法。

可以先载入一个样例程序（在本模拟器所在的文件夹下的“样例程序”文件夹中），然后分别以单步执行一个周期、执行多个周期、连续执行、设置断点等方式运行程序，观察程序的执行情况，观察 CPU 中寄存器和存储器的内容的变化，特别是流水寄存器内容的变化。

4. 选择配置菜单中的“流水方式”，使模拟器工作于流水方式下。
5. 观察程序在流水线中的执行情况，步骤如下：
 - (1) 选择 MIPSsim 的“文件”→“载入程序”选项来加载 pipeline.s（在模拟器所在文件夹下的“样例程序”文件夹中）。
 - (2) 关闭定向功能。这是通过在“配置”→“定向”（使该项前面没有“√”号）来实现的。
 - (3) 用单步执行一周期的方式（“执行”菜单中）或用 F7 执行该程序，观察每一周期中，各段流水寄存器内容的变化、指令的执行情况（“代码”窗口）以及时钟周期图。
 - (4) 当执行到第 13 个时钟周期时，各段分别正在处理的指令是：

IF: _____

ID: _____

EX: _____

MEM: _____

WB: _____

画出这时的时钟周期图。

6. 这时各流水寄存器中的内容为：

IF/ID.IR: _____

IF/ID.NPC: _____

ID/EX.A: _____

ID/EX.B: _____

ID/EX.Imm: _____

ID/EX.IR: _____

EX/MEM.ALUo: _____

EX/MEM.IR: _____

MEM/WB.LMD: _____

MEM/WB.ALUo: _____

MEM/WB.IR: _____

7. 观察和分析结构冲突对 CPU 性能的影响，步骤如下：

(1) 加载 `structure_hz.s`（在模拟器所在文件夹下的“样例程序”文件夹中）。

(2) 执行该程序，找出存在结构冲突的指令对以及导致结构冲突的部件。

(3) 记录由结构冲突引起的停顿时钟周期数，计算停顿时钟周期数占总执行周期数的百分比；

(4) 把浮点加法器的个数改为 4 个。

- (5) 再次重复上述 (1) ~ (3) 的工作。
- (6) 分析结构冲突对 CPU 性能的影响, 讨论解决结构冲突的方法。
- 8. 观察数据冲突并用定向技术来减少停顿, 步骤如下:
 - (1) 全部复位。
 - (2) 加载 data_hz.s (在模拟器所在文件夹下的“样例程序”文件夹中)。
 - (3) 关闭定向功能。这是通过在“配置”→“定向”(使该项前面没有“√”号)来实现的。
 - (4) 用单步执行一个周期的方式(F7)执行该程序, 同时查看时钟周期图, 列出在什么时刻发生了 RAW (先写后读) 冲突。
 - (5) 记录数据冲突引起的停顿时钟周期数以及程序执行的总时钟周期数, 计算停顿时钟周期数占总执行周期数的百分比。
 - (6) 复位 CPU。
 - (7) 打开定向功能。这是通过在“配置”→“定向”(使该项前面有一个“√”号)来实现的。
 - (8) 用单步执行一周期的方式(F7)执行该程序, 同时查看时钟周期图, 列出在什么时刻发生了 RAW (先写后读) 冲突, 并与(3)的结果进行比较;
 - (9) 记录数据冲突引起的停顿时钟周期数以及程序执行的总时钟周期数。计算采用定向技术后性能提高的倍数。

A.3 实验三 指令调度和延迟分支

A.3.1 实验目的

1. 加深对指令调度技术的理解。
2. 加深对延迟分支技术的理解。
3. 熟练掌握用指令调度技术来解决流水线中的数据冲突的方法。
4. 进一步理解指令调度技术对 CPU 性能的改进。
5. 进一步理解延迟分支技术对 CPU 性能的改进。

A.3.2 实验平台

实验平台采用指令级和流水线操作级模拟器 MIPSsim。
环境的建立：见 A.0。

A.3.3 实验内容和步骤

首先要掌握 MIPSsim 模拟器的使用方法。详见附录 B。

1. 启动 MIPSsim。
2. 根据教材中关于流水线各段操作的描述，进一步理解流水线窗口中各段的功能，掌握各流水寄存器的含义（双击各段，就可以看到各流水寄存器的内容）。
3. 选择“配置”→“流水方式”选项，使模拟器工作于流水方式下。
4. 用指令调度技术解决流水线中的结构冲突与数据冲突。
 - (1) 启动 MIPSsim。
 - (2) 用 MIPSsim 的“文件”→“载入程序”选项来加载 schedule.s（在模拟器所在文件夹下的“样例程序”文件夹中）。
 - (3) 关闭定向功能。这是通过在“配置”菜单中关闭“定向”（使该项前面没有“√”号）来实现的。
 - (4) 执行所载入的程序。通过查看统计数据 and 时钟周期图，找出并记录程序执行过程中各种冲突发生的次数、发生冲突的指令组合，以及程序执行的总时钟周期数。
 - (5) 采用指令调度技术对程序进行指令调度，消除冲突。将调度后的程序保持到 after-schedule.s 中。
 - (6) 载入 after-schedule.s。
 - (7) 执行该程序，观察程序在流水线中的执行情况，记录程序执行的总时钟周期数；
 - (8) 根据记录结果，比较调度前和调度后的性能。论述指令调度对于提高 CPU 性能的作用。
5. 用延迟分支减少分支指令对性能的影响。
 - (1) 启动 MIPSsim。
 - (2) 载入 branch.s。

- (3) 关闭延迟分支功能。这是通过在“配置”→“延迟槽”选项来实现的。
- (4) 执行该程序。观察并记录发生分支延迟的时刻。
- (5) 记录执行该程序所花的总时钟周期数。
- (6) 假设延迟槽为一个，对 `branch.s` 进行指令调度，然后保存到“`delayed-branch.s`”中。
- (7) 载入 `delayed-branch.s`。
- (8) 打开延迟分支功能。
- (9) 执行该程序，观察其时钟周期图。
- (10) 记录执行该程序所花的总时钟周期数。
- (11) 对比上述两种情况下的时钟周期图。
- (12) 根据记录结果，比较没采用延迟分支和采用了延迟分支的性能。论述延迟分支对于提高 CPU 性能的作用。

A.4 实验四 Cache 性能分析

A.4.1 实验目的

1. 加深对 Cache 的基本概念、基本组织结构以及基本工作原理的理解。
2. 掌握 Cache 容量、相联度、块大小对 Cache 性能的影响。
3. 掌握降低 Cache 不命中率的各种方法以及这些方法对提高 Cache 性能的好处。
4. 理解 LRU 与随机法的基本思想以及它们对 Cache 性能的影响。

A.4.2 实验平台

实验平台采用 Cache 模拟器 MyCache。
环境的建立：见 A.0。

A.4.3 实验内容及步骤

首先要掌握 MyCache 模拟器的使用方法（见 A.4.4 节）。

A.4.3.1 Cache 容量对不命中率的影响

1. 启动 MyCache。
2. 用鼠标点击“复位”按钮，把各参数设置为默认值。
3. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后点击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。
4. **选择不同的 Cache 容量**，包括：2KB, 4KB, 8KB, 16KB, 32KB, 64KB, 128KB, 256KB，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表 A.4.1 中记录各种情况下的不命中率。

表 A.4.1 不同容量下 Cache 的不命中率

Cache 容量 (KB)	2	4	8	16	32	64	128	256
不命中率								

地址流文件名：_____

5. 以容量为横坐标，画出不命中率随 Cache 容量变化而变化的曲线。并指明地址流文件名。
6. 根据该模拟结果，你能得出什么结论？

A.4.3.2 相联度对不命中率的影响

1. 用鼠标单击“复位”按钮，把各参数设置为默认值。此时的 Cache 容量为 64KB。
2. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后单击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。

3. 选择不同的 **Cache 相联度**，包括：直接映象，2 路，4 路，8 路，16 路，32 路，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表 A.4.2 中记录各种情况下的不命中率。

表 A.4.2 当容量为 64KB 时，不同相联度下 Cache 的不命中率

相联度	1	2	4	8	16	32
不命中率						

地址流文件名：_____

4. 把 Cache 的容量设置为 256KB，重复(3)的工作，并填写表 A.4.3。

表 A.4.3 当容量为 256KB 时，不同相联度下 Cache 的不命中率

相联度	1	2	4		8	16	32
不命中率							

地址流文件名：_____

5. 以相联度为横坐标，画出在 64KB 和 256KB 的情况下不命中率随 Cache 相联度变化而变化的曲线。并指明地址流文件名。
6. 根据该模拟结果，你能得出什么结论？

A.4.3.3 Cache 块大小对不命中率的影响

1. 用鼠标单击“复位”按钮，把各参数设置为默认值。
2. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后单击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。
3. 选择不同的 **Cache 块大小**，包括：16B，32B，64B，128B，256B，对于 **Cache 的各种容量**，包括：2KB，8KB，32KB，128KB，512KB，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表 A.4.4 中记录各种情况下的不命中率。

表 A.4.4 各种块大小情况下 Cache 的不命中率

块大小 (B)	Cache 容量 (KB)				
	2	8	32	128	512
16					
32					
64					
128					
256					

地址流文件名：_____

4. 分析 Cache 块大小对不命中率的影响。

A.4.3.4 替换算法对不命中率的影响

1. 用鼠标单击“复位”按钮，把各参数设置为默认值。
2. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后单击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。
3. 对于不同的替换算法、Cache 容量和相联度，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表 A.4.5 中记录各种情况下的不命中率。

表 A.4.5 LRU 和随机替换法的不命中率的比较

Cache 容量	相 联 度					
	2 路		4 路		8 路	
	LRU	随机算法	LRU	随机算法	LRU	随机算法
16KB						
64KB						
256KB						
1MB						

地址流文件名：_____

4. 分析不同的替换算法对 Cache 不命中率的影响。

A.4.4 MyCache 模拟器使用方法

1. 启动模拟器：用鼠标双击 MyCache.exe。
2. 系统会打开一个操作界面。该界面的左边为设置模拟参数区域，右边为模拟结果显示区域。如图 A.4.1 所示。
3. 可以设置的参数包括：是统一 Cache 还是分离 Cache，Cache 的容量，块大小，相联度，替换算法，预取策略，写策略，写不命中时的调块策略。可以直接从列表里选择。
4. 访问地址可以选择来自地址流文件，也可以选择手动输入。如果是前者，则可以通过点击“浏览”按钮，从模拟器所在文件夹下面的“地址流”文件夹中选取地址流文件（.din 文件），然后进行执行。执行的方式可以是步进，也可以是一次执行到底。如果选择手动输入，就可以在“执行控制”区域中输入块地址，然后点击“访问”按钮。系统会在界面的右边显示访问类型、地址、块号以及块内地址。
5. 模拟结果包括：
 - （1）访问总次数，总的命中次数，总的命中率；
 - （2）读指令操作的次数，其命中次数及其命中率；
 - （3）读数据操作的次数，其命中次数及其命中率；
 - （4）写数据操作的次数，其命中次数及其命中率；
 - （5）手动输入单次访问的相关信息。

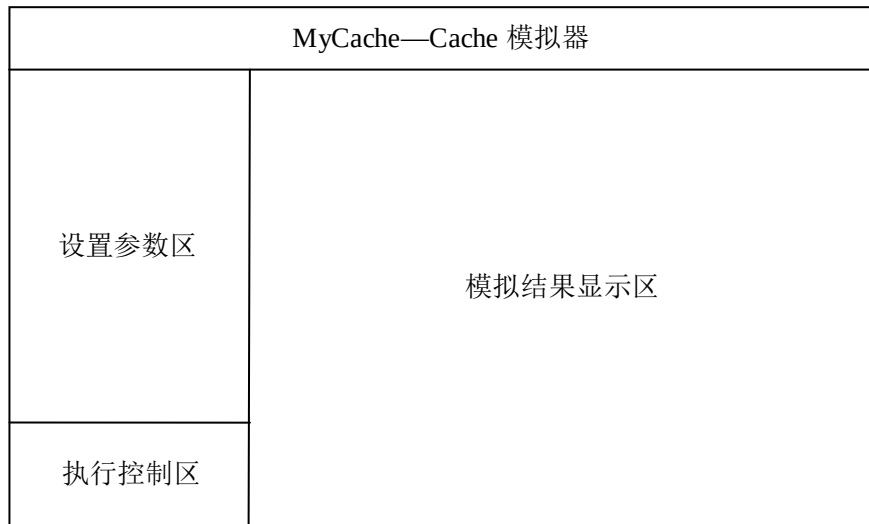


图 A.4.1 MyCache 模拟器的操作界面示意图

A.5 实验五 Tomasulo 算法

A.5.1 实验目的

1. 加深对指令级并行性及其开发的理解。
2. 加深对 Tomasulo 算法的理解。
3. 掌握 Tomasulo 算法在指令流出、执行、写结果各阶段对浮点操作指令以及 load 和 store 指令进行什么处理。
4. 掌握采用了 Tomasulo 算法的浮点处理部件的结构。
5. 掌握保留站的结构。
6. 给定被执行代码片段，对于具体某个时钟周期，能够写出保留站、指令状态表以及浮点寄存器状态表内容的变化情况。

A.5.2 实验平台

实验平台采用 Tomasulo 算法模拟器。
环境的建立：见 A.0。

A.5.3 实验内容及步骤

首先要掌握 Tomasulo 模拟器的使用方法。（见 A.5.4 节）

1. 假设浮点功能部件的延迟时间为加减法 2 个时钟周期，乘法 10 个时钟周期，除法 40 个时钟周期，Load 部件 2 个时钟周期。

（1）对于下面的代码段，给出当指令 MUL.D 即将确认时，保留站、load 缓冲器以及寄存器状态表中的内容。

```
L.D      F6, 24 (R2)
L.D      F2, 12 (R3)
MUL.D    F0, F2, F4
SUB.D    F8, F6, F2
DIV.D    F10, F0, F6
ADD.D    F6, F8, F2
```

（2）按步进方式执行上述代码，利用模拟器的“小三角按钮”的对比显示功能，观察每一个时钟周期前后各信息表中内容的变化情况。

2. 对于与上面相同的延迟时间和代码段。

- （1）给出在第 3 个时钟周期时，保留站的内容。
- （2）步进 5 个时钟周期，给出这时保留站、load 缓冲器以及寄存器状态表中的内容。
- （3）再步进 10 个时钟周期，给出这时保留站、load 缓冲器以及寄存器状态表中的内容。

3. 假设浮点功能部件的延迟时间为加减法 3 个时钟周期，乘法 8 个时钟周期，除法 40

个时钟周期。自己编写一段程序（要在实验报告中给出），重复上述步骤 2 的工作。

A.5.4 Tomasulo 算法模拟器使用方法

1. 设置指令和参数

本模拟器最多可以模拟 10 条指令。可以在“指令”区选择和设置所要的指令。“指令”区如图 A.5.1 所示。

指令

L. D	▼	F8	▼	Z1	▼	R3	▼
L. D	▼	F4	▼	16	▼	R4	▼
MULT. D	▼	F2	▼	F4	▼	F6	▼
SUB. D	▼	F10	▼	F8	▼	F4	▼
DIV. D	▼	F12	▼	F2	▼	F8	▼
ADD. D	▼	F8	▼	F10	▼	F4	▼
NOP	▼	Null	▼	Null	▼	Null	▼
NOP	▼	Null	▼	Null	▼	Null	▼
NOP	▼	Null	▼	Null	▼	Null	▼
NOP	▼	Null	▼	Null	▼	Null	▼

图 A.5.1 “指令”区

你可以从下拉框中选择指令，供选择的指令有以下 5 种：

- (1) L.D 指令：从主存读取一个双精度浮点数；
- (2) ADD.D：双精度浮点加法指令；
- (3) SUB.D：双精度浮点减法指令；
- (4) MULT.D：双精度浮点乘法指令；
- (5) DIV.D：双精度浮点除法指令。

指令的各参数也可以从各自的下拉框中选择。

你还可以在窗口的右上区域设置各部件的执行时间（时钟周期数），如图 A.5.2 所示。

功能部件的执行时间

Load	2	加/减	2
乘法	10	除法	40

执行 复位

图 A.5.2 设置功能部件时间

其中“复位”的作用是使所有设置恢复为默认值。

2. 执行

点击“执行”按钮，就进入执行状态。你可以用中间的按钮来控制指令的执行，包括“步进”、“退 1 步”、“前进 5 个周期”、“后退 5 个周期”、“执行到底”、“退出”等。还可以用“go”按钮直接跳转到你所指定的时钟周期。如果想修改被执行的代码，按“退出”按钮，即可回到设置指令和参数页面。

向前执行后，状态表中抹色的字段表示其内容发生了变化。

3. 对比状态表

每一个状态表的右上角外侧都有一个小三角，用鼠标左键点击它，会弹出该表在上一个时钟周期的内容。这是为了让你通过对比来了解哪些内容发生了变化。在弹出表以外的区域再次点击鼠标，就可以将其收回。

4. 各个表的内容

(1) 指令状态表

指令状态表如图 A.5.3 所示。它列出了各指令什么时候执行到了哪一步。其中的数字表示时钟周期，“~”表示时钟周期期间。例如，图 A.5.3 中的 2~3 表示在第 2 到第 3 个时钟周期（含第 3 个），第一条 L.D 指令是在“执行”这一步。

指令状态			
指令	流出	执行	写结果
L.D F8, 21(R3)	1	2~3	
L.D F4, 16(R4)	2	3~	
MULT.D F2, F4, F6	3		
SUB.D F10, F8, F4			
DIV.D F12, F2, F8			
ADD.D F8, F10, F4			

图 A.5.3 指令状态表

其中抹色的区域表示最近一个时钟周期其内容发生了变化。下同。

(2) 保留站

保留站的内容如图 A.5.4 所示。

保留站							
Time	名称	Busy	Op	Vj	Vk	Qj	Qk
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

图 A.5.4 保留站

其中各字段的名称和含义如下：

Time: 表示相应的保留站还要执行多少个时钟周期；

名称: 保留站的名称。用于唯一地标识相应的保留站；

Op: 要对源操作数进行的操作；

Qj, Qk: 将产生源操作数的保留站名称。其值等于 0 表示操作数已经就绪且在 Vj 或 Vk 中，或者不需要操作数。

Vj, Vk: 源操作数的值。对于每一个操作数来说，V 和 Q 字段只有一个有效。

Busy: 为“**Yes**”表示该保留站“忙”。

(3) Load 部件

Load 部件的内容如图 A.5.5 所示。它按队列方式工作，每次处理新的访存都是从队列头部取走一条。

Load部件			
名称	Busy	地址	值
Load1	Yes	21	
Load2	No		
Load3	No		

图 A.5.5 Load 部件

其中各字段的含义如下：

名称: 相应单元的名称（标识）；

Busy: “忙”标志，为“**Yes**”表示已被占用；

地址: 访存的有效地址；

值: 存放从存储器读来的数据。

(4) 寄存器

寄存器的内容如图 A.5.6 所示。

寄存器										
字段	F0	F2	F4	F6	F8	F10	F12	F14	F16	F18
Qi					Load1					

图 A.5.6 寄存器的内容

各字段的含义如下：

Qi: 寄存器状态，用于存放将把结果写入该寄存器的保留站的站号。为 0 表示当前没有正在执行的指令要写入该寄存器，也即该寄存器中的内容就绪。

值: 寄存器的值。

当上述表中的内容写不下时，模拟器会采用缩写的方法。这时，在上面中间的区域中会显示缩写及其值。

A.6 实验六 再定序缓冲（ROB）工作原理

A.6.1 实验目的

1. 加深对指令级并行性及其开发的理解。
2. 加深对基于硬件的前瞻执行的理解。
3. 掌握 ROB 在流出、执行、写结果、确认 4 个阶段所进行的操作。
4. 掌握 ROB 缓冲器的结构。
5. 给定被执行代码片段，对于具体某个时钟周期，能写出保留站、ROB 以及浮点寄存器状态表内容的变化情况。

A.6.2 实验平台

实验平台采用再定序缓冲 ROB 模拟器。
环境的建立：见 A.0。

A.6.3 实验内容及步骤

首先要掌握 ROB 模拟器的使用方法（见 A.6.4 节）。

1. 假设浮点功能部件的延迟时间为加法 2 个时钟周期，乘法 10 个时钟周期，除法 40 个时钟周期，Load 部件 2 个时钟周期。

（1）对于下面的代码段，给出当指令 MUL.D 即将确认时保留站、ROB 以及浮点寄存器状态表的内容。

```
L.D      F6, 24 (R2)
L.D      F2, 12 (R3)
MUL.D    F0, F2, F4
SUB.D    F8, F6, F2
DIV.D    F10, F0, F6
ADD.D    F6, F8, F2
```

（2）按步进方式执行上述代码，利用模拟器的“小三角按钮”的对比显示功能，观察每一个时钟周期前后保留站、ROB 以及浮点寄存器状态表的内容的变化情况。

2. 对于与上面相同的延迟时间和代码段：

- （1） 给出在第 5 个时钟周期时，保留站的内容。
- （2） 步进 5 个时钟周期，ROB 的内容有哪些变化？
- （3） 再步进 5 个时钟周期，给出这时保留站、ROB 以及浮点寄存器状态表的内容。

3. 假设浮点功能部件的延迟时间为加减法 3 个时钟周期，乘法 8 个时钟周期，除法 40 个时钟周期。自己编写一段程序（要在实验报告中给出），重复上述步骤 2 的工作。

A.6.4 ROB 模拟器的使用方法

1. 设置指令和参数

本模拟器最多可以模拟 10 条指令。可以在“指令”区选择和设置所要的指令。“指令”区如图 A.6.1 所示。

指令

L.D	▼	F8	▼	21	▼	R3	▼
L.D	▼	F4	▼	16	▼	R4	▼
MULT.D	▼	F2	▼	F4	▼	F6	▼
SUB.D	▼	F10	▼	F8	▼	F4	▼
DIV.D	▼	F12	▼	F2	▼	F8	▼
ADD.D	▼	F8	▼	F10	▼	F4	▼
NOP	▼	Null	▼	Null	▼	Null	▼
NOP	▼	Null	▼	Null	▼	Null	▼
NOP	▼	Null	▼	Null	▼	Null	▼
NOP	▼	Null	▼	Null	▼	Null	▼

图 A.6.1 “指令”区

你可以从下拉框中选择指令，供选择的指令有以下 5 种：

- (1) L.D 指令：从主存读取一个双精度浮点数；
- (2) ADD.D：双精度浮点加法指令；
- (3) SUB.D：双精度浮点减法指令；
- (4) MULT.D：双精度浮点乘法指令；
- (5) DIV.D：双精度浮点除法指令。

指令的各参数也可以从各自的下拉框中选择。

你还可以在窗口的右上区域设置各部件的执行时间（时钟周期数），如图 A.6.2 所示。

功能部件的执行时间

Load	2	加/减	2
乘法	10	除法	40

执行 复位

图 A.6.2 设置功能部件时间

其中“复位”的作用是使所有设置恢复为默认值。

2. 执行

点击“执行”按钮，就进入执行状态。你可以用中间的按钮来控制指令的执行，包括“步进”、“退 1 步”、“前进 5 个周期”、“后退 5 个周期”、“执行到底”、“退出”等。还可以用“go”按钮直接跳转到你所指定的时钟周期。如果想修改被执行的代码，按“退出”按钮，即可回

到设置指令和参数页面。
向前执行后，状态表中抹色的字段表示其内容发生了变化。

3. 对比状态表

每一个状态表的右上角外侧都有一个小三角，用鼠标左键点击它，会弹出该表在上一个时钟周期的内容。这是为了让你通过对比来了解哪些内容发生了变化。在弹出表以外的区域再次点击鼠标，就可以将其收回。

4. 各个表的内容

(1) 指令状态表

指令状态表如图 A.6.3 所示。它列出了各指令什么时候执行到了哪一步。其中的数字表示时钟周期，“~”表示时钟周期期间。例如，图 A.6.3 中的 2~3 表示在第 2 到第 3 个时钟周期，第一条 L.D 指令是在“执行”这一步。

指令状态				
指令	流出	执行	写结果	确认
L.D F8, 21 (R3)	1	2~3		
L.D F4, 16 (R4)	2	3~		
MULT.D F2, F4, F6	3			
SUB.D F10, F8, F4				
DIV.D F12, F2, F8				
ADD.D F8, F10, F4				

图 A.6.3 指令状态表

其中抹色的区域表示最近一个时钟周期其内容发生了变化。下同。

(2) 再定序缓冲器 ROB

ROB 如图 A.6.4 所示。它按队列方式工作，其中各字段的意义如下：

- ◆ 标记：用于给出队列的头和尾；
- ◆ 项号：给出各项的编号；
- ◆ Busy：“忙”标志，指出相应的行是否已占用；
- ◆ 指令：给出是什么指令占用该行；
- ◆ 目的地：指出结果写到哪里去；
- ◆ 值：暂时存放相应指令的计算结果，在该指令被确认时，将被写到目的地。

再定序缓冲器 (ROB)

标记	项号	Busy	指令	目的地	值
HEAD	#1	Yes	L.D F8, 21(R3)	F8	
	#2	Yes	L.D F4, 16(R4)	F4	
	#3	Yes	MULT.D F2, F4, F6	F2	
TAIL	#4	No			
	#5	No			
	#6	No			
	#7	No			
	#8	No			
	#9	No			
	#10	No			

图 A.6.4 再定序缓冲器 ROB

(3) 保留站

保留站的内容如图 A.6.5 所示。

保留站

Time	名称	Busy	Op	Vj	Vk	Qj	Qk	目的地
	Add1	No						
	Add2	No						
	Add3	No						
	Mult1	Yes	MULT.D		R[F6]	#2		#3
	Mult2	No						

图 A.6.5 保留站

其中各字段的名称和意义与图 A.5.4 中的相同。不过，这里增加了一个字段：目的地。它指出相应部件的运算结果要暂时存放到 ROB 的第几号单元。

(4) Load 缓冲器

Load 缓冲器的内容如图 A.6.6 所示。它按队列方式工作，每次处理新的访存都是从队列头部取走一条。

Load 缓冲器

名称	Busy	地址	目的地	值
Load1	Yes	21+R[R3]	#1	M[21+R[R3]]
Load2	Yes	16+R[R4]	#2	
Load3	No			

图 A.6.6 Load 缓冲器

该缓冲器各字段的意义如下：

- ◆ 名称：相应单元的名称；
- ◆ Busy：“忙”标志，为“Yes”表示已被占用；
- ◆ 地址：访存的有效地址；
- ◆ 目的地：指出从存储器读来的数据要暂时存放到 ROB 的第几号单元；
- ◆ 值：存放从存储器读来的数据。

(5) 寄存器

寄存器的内容如图 A.6.7 所示。

寄存器											
字段	F0	F2	F4	F6	F8	F10	F12	F14	F16	F18	F20
ROB项号		#3	#2		#1						
Busy	No	Yes	Yes	No	Yes	No	No	No	No	No	No
值											

图 A.6.7 寄存器的内容

该缓冲器各字段的意义如下：

- ◆ Busy: “忙”标志，为“**Yes**”表示将有指令要对该寄存器写入数据；
- ◆ ROB 项号：指出它在等哪个 ROB 项的数据。当那个 ROB 项中的指令被确认且其值已经就绪时，那个数据将被写入该寄存器；
- ◆ 值：寄存器的值。

当上述表中的内容写不下时，模拟器会采用缩写的方法。这时，在屏幕上面中间的区域中会显示缩写及其值。

A.7 实验七 多 Cache 一致性——监听协议

A.7.1 实验目的

1. 加深对多 Cache 一致性的理解。
2. 进一步掌握解决多 Cache 一致性的监听协议的基本思想。
3. 掌握在各种情况下，监听协议是如何工作的。能给出要进行什么样的操作以及状态的变化情况。

A.7.2 实验平台

实验平台采用多 Cache 一致性监听协议模拟器（附书光盘中提供）。
环境的建立：见 A.0。

A.7.3 实验内容及步骤

首先要掌握该模拟器的使用方法（见 A.7.4 节）。

1. 对于以下访问序列，写出监听协议所进行的操作。

所进行的访问	是否发生 替换	是否发生 写回	监听协议所进行的操作
CPU A 读第 5 块			
CPU B 读第 5 块			
CPU C 读第 5 块			
CPU B 写第 5 块			
CPU D 读第 5 块			
CPU B 写第 21 块			
CPU A 写第 23 块			
CPU C 写第 23 块			
CPU B 读第 29 块			
CPU B 写第 5 块			

2. 自己编写一个访问序列，写出监听协议所进行的操作。

所进行的访问	是否发生 替换	是否发生 写回	监听协议所进行的操作

3. 根据上述结果，画出相关的状态转换图。

A.7.4 监听协议模拟器使用方法

该模拟器模拟 4 个 CPU（A、B、C、D）访存的工作过程。每个 CPU 中都有一个 Cache，该 Cache 包含 4 个块，其块地址为 0~3。集中共享存储器中有 32 个块，其块地址为 0~31。每个块的状态用色块来表示，其中灰色表示“无效”状态，淡青色表示“共享”，橘红色表示“独占”。

对于每个 CPU，可以指定所要进行的访问是读还是写（从列表中选），并在输入框中输入所要访问的主存块号，然后用鼠标点击在其右边的标有↓的按钮，模拟器就将开始演示该访问的工作过程。

该模拟器的主菜单有 4 个：配置，控制，统计，帮助。

1. 配置

该菜单用于进行配置参数的显示与设置。你可以修改动画播放速度：把游标往右边拖拽可提高播放速度，往左边拖拽可降低播放速度。你还可以选择是否进行优化传块。优化传块是指当要访问的块在某个 Cache 中，且处于独占状态时，可以不用等该块写回主存后再从主存调块，而是可以直接将该块传送给发出访问请求的结点。

本模拟器采用直接映象方法和写回法。

2. 控制

可以通过该菜单中的选项来控制模拟器的执行。该菜单下有以下 3 个选项：单步执行、连续执行、复位。

(1) 单步执行

选用该方式后，点击鼠标或点击左上角的“步进”按钮，都会使模拟器前进一步。

(2) 连续执行

选用该方式后，用鼠标点击标有↓的按钮，模拟器会连续演示一次访存的整个过程，直至该访问结束。

(3) 复位

使模拟器复位，回到初始状态。

也可以通过点击窗口内左上角的选项和按钮来控制模拟器的执行，其功能与上述菜单选项相同。

3. 统计

该菜单用于显示模拟器的统计结果，包括各处理机的访问次数、命中次数、不命中次数以及命中率。

4. 帮助

该菜单下有“关于”和“使用说明”两个选项。

A.8 实验八 多 Cache 一致性——目录协议

A.8.1 实验目的

1. 加深对多 Cache 一致性的理解。
2. 进一步掌握解决多 Cache 一致性的目录协议的基本思想。
3. 掌握在各种情况下，目录协议是如何工作的。能给出要进行什么样的操作以及状态的变化情况。

A.8.2 实验平台

实验平台采用多 Cache 一致性目录协议模拟器。
环境的建立：见 A.0。

A.8.3 实验内容及步骤

首先要掌握该模拟器的使用方法（见 A.8.4 节）。

1. 对于以下访问序列，写出目录协议所进行的操作。

所进行的访问	目录协议所进行的操作
CPU A 读第 6 块	
CPU B 读第 6 块	
CPU D 读第 6 块	
CPU B 写第 6 块	
CPU C 读第 6 块	
CPU D 写第 20 块	
CPUA 写第 20 块	
CPU D 写第 6 块	
CPU A 读第 12 块	

2. 自己编写一个访问序列，写出目录协议所进行的操作。

所进行的访问	是否发生 替换	是否发生 写回	目录协议所进行的操作

3. 根据上述结果，画出相关的状态转换图（仅画出与上表相关的部分）。

A.8.4 目录协议模拟器使用方法

该模拟器模拟 4 个 CPU（A、B、C、D）访存的工作过程。每个 CPU 中都有一个 Cache，该 Cache 包含 4 个块，其块地址为 0~3。分布式存储器中有 32 个块，其块地址为 0~31。Cache 中每个块的状态用色块来表示，其中灰色表示“无效”状态，淡青色表示“共享”，橘红色表示“独占”。主存中块的状态由其右边的目录项的颜色来表示，未缓冲状态由黄色来表示，其他两种状态同 Cache 块。

对于每个 CPU，可以指定所要进行的访问是读还是写（从列表中选），并在输入框中输入所要访问的主存块号，然后用鼠标点击在其右边的标有↓的按钮，模拟器就将开始演示该访问的工作过程。

该模拟器的主菜单有 4 个：配置，操作，统计，帮助。

1. 配置

该菜单用于进行配置参数的显示与设置。你可以修改动画播放速度：把游标往右边拖拽可提高播放速度，往左边拖拽可降低播放速度。你还可以选择是否进行优化传块。优化传块是指当要访问的块在某个 Cache 中，且处于独占状态时，可以不用等该块写回主存后再从主存调块，而是可以直接将该块传送给发出访问请求的结点。

本模拟器采用直接映象方法和写回法。

2. 操作

可以通过该菜单中的选项来控制模拟器的执行。该菜单下有 3 个选项：单步执行、连续执行、复位。

(1) 单步执行

选用该方式后，敲任意键、点击鼠标或点击左上角的“步进”按钮，都会使模拟器前进一步。

(2) 连续执行

选用该方式后，用鼠标点击标有 ↓ 的按钮，模拟器会连续演示一次访存的整个过程，直至该访问结束。

(3) 复位

使模拟器复位，回到初始状态。

3. 统计

该菜单用于显示模拟器的统计结果，包括各处理机的访问次数、命中次数、不命中次数以及命中率。

4. 帮助

该菜单下有“关于”和“使用说明”两个选项。

A.9 实验九 大作业：编写一个指令级模拟器

A.9.1 实验目的

1. 掌握指令级模拟器的设计与实现。
2. 进一步理解和掌握 16 条 MIPS 指令的操作语义。
3. 进一步熟悉 MIPS 体系结构。

A.9.2 实验平台

实验平台采用预先编制好的指令级模拟器(模拟 16 条指令的执行,以下称为 MIPS16)。MIPS16 只完成了一半,并提供了源代码 `simulator.c`, 本实验的任务是编写完成其余的代码,放在 `executor_学号.c` 中,然后将它们一起编译和链接,形成完整的 MIPS16。MIPS16 完成的辅助功能见附录 C。

所有相关的文件都放在阿里云实验机的桌面上的 `comparch-simulator` 目录里的 MIPS 子目录下。

A.9.3 实验内容和步骤

本实验是要用 C 语言编写一个精简了的指令集(16 条指令)的模拟器。该模拟器的数据结构的定义以及辅助功能已经完成。详见附录。

该 16 条指令如下:

ADD.D, AND, BNE, BNEZ, DADD, DADDIU, DIV.D, DMULT, DSUB, L.D, LW, MUL.D, S.D, SUB.D, SW, XOR。

这些指令的功能与格式见附录 C。

实验步骤如下:

1. 进入桌面上 `comparch-simulator` 目录下的 MIPS 子目录。
2. 仔细阅读 `simulator.c` 源程序,读懂所有代码。特别是数据结构的定义。
3. 启动 VSCode (双击),然后用 C 语言写出上述 16 条指令的解释程序。放入到“`executor_学号.c`”文件中的 `execute()` 函数里。
4. 将 `simulator.c` 和“`executor_学号.c`”两个程序一起编译链接,形成可执行文件“`MIPS16_学号.exe`”。即执行命令:

```
gcc simulator.c executor_学号.c -o MIPS16_学号
```
5. 编写测试程序“`test_学号.s`”,其中要全面测试各条指令的模拟结果(格式可参考附录 C 中的样例程序 `sample.s`)。
6. 启动 MIPS16 (双击“`MIPS16_学号.exe`”)。
7. 在 MIPS16 中载入(用 `load` 命令)“`test_学号.s`”。
8. 以单步执行指令的方式执行代码,观察每一条指令执行前后寄存器和存储器内容的变化情况,直至全部指令执行完毕。
9. 执行“`dump output_学号.txt`”命令,生成结果文件: `output_学号.txt`
10. 把“`executor_学号.c`”、“`MIPS16_学号.exe`”、“`test_学号.s`”和“`output_学号.txt`”一共 4 个文件提交给老师。

附录 B MIPSsim 使用手册

B.1 启动模拟器

双击 MIPSsim.exe，即可启动该模拟器。

模拟器启动时，自动将自己初始化为默认状态。所设置的默认值为：

- ◆ 所有通用寄存器和浮点寄存器为全 0；
- ◆ 内存清零；
- ◆ 流水寄存器为全 0；
- ◆ 清空时钟图、断点、统计数据；
- ◆ 内存大小为 4096 字节；
- ◆ 载入起始地址为 0；
- ◆ 浮点加法、乘法、除法部件的个数均为 1；
- ◆ 浮点加法、乘法、除法运算延迟分别为 6、7、10 个时钟周期；
- ◆ 采用流水方式；
- ◆ 不采用定向机制；
- ◆ 不采用延迟槽；
- ◆ 采用符号地址；
- ◆ 采用绝对周期计数。

当模拟器工作在非流水方式下（配置菜单中的“流水方式”前没有“√”号）时，下面叙述中有关流水段的内容都没有意义，应该忽略之。

B.2 MIPSsim 的窗口

在流水方式下，模拟器主界面中共有 7 个子窗口，它们是：代码窗口、寄存器窗口、流水线窗口、时钟周期图窗口、内存窗口、统计窗口和断点窗口。每一个窗口都可以被收起（变成小图标）、展开、拖动位置和放大/缩小。当要看窗口的全部内容时，可以将其放大到最大。

在非流水方式下，只有代码窗口、寄存器窗口、内存窗口和断点窗口。

B.2.1 代码窗口

代码窗口给出内存中代码的列表，每条指令占一行，按地址顺序排列。每行有 5 列（当全部显示时）：地址、断点标记、指令的机器码、流水段标记和符号指令。如图 B.1 所示。

地址	断点标记	机器码	流水段	符号指令
0x00000000		0x8C01003C		LW \$1,60(\$0)
0x00000004		0x04310004	WB	BGEZAL \$1,4
0x00000008		0x00000000	MEM	SLL \$0,\$0,0
0x0000000C	B.MEM	0xAC02003C		SW \$2,60(\$0)
0x00000010	B.IF	0x00000034		TEQ \$0,\$0
0x00000014		0x00000000		SLL \$0,\$0,0
0x00000018		0x00201020	EX	ADD \$2,\$1,\$0
0x0000001C		0x2021FFFF	ID	ADDI \$1,\$1,-1
0x00000020		0x10200004	IF	BEQ \$1,\$0,4
0x00000024		0x00000000		SLL \$0,\$0,0
0x00000028		0x70411002		MUL \$2,\$2,\$1
0x0000002C		0x1000FFFB		BEQ \$0,\$0,-5
0x00000030		0x00000000		SLL \$0,\$0,0
0x00000034	B.ID	0x03E00008		JR \$31
0x00000038		0x00000000		SLL \$0,\$0,0
0x0000003C		0x00000005		
0x00000040		0x00000000		SLL \$0,\$0,0
0x00000044		0x00000000		SLL \$0,\$0,0
0x00000048		0x00000000		SLL \$0,\$0,0
0x0000004C		0x00000000		SLL \$0,\$0,0
0x00000050		0x00000000		SLL \$0,\$0,0
0x00000054		0x00000000		SLL \$0,\$0,0
0x00000058		0x00000000		SLL \$0,\$0,0
0x0000005C		0x00000000		SLL \$0,\$0,0

图 B.1 代码窗口

图中不同抹色的行代表相应的指令所处的执行段。黄色代表 IF 段，绿色代表 ID 段，红色代表 EX 段，青色代表 MEM 段，棕色代表 WB 段。

该窗口中各列的含义如下：

- ◆ 地址：以 16 进制的形式给出。内存是按字节寻址的，每条指令占 4 个字节。当采用符号地址时，会在相应的位置给出汇编程序中出现的标号。
- ◆ 断点标记：如果在该指令处设有断点，则显示相应的标记。断点标记的形式为 B.X（X 为段名），表示该断点是设置在该指令的“X”段。例如，若某行的断点标记为“B.EX”，则表示在该指令的 EX 段设置了断点。
当模拟器工作在非流水方式下时，断点的标记为 B。
- ◆ 机器码：该行所对应的指令的十六进制机器码。若该行无指令，则仅仅显示 4 字节数据；
- ◆ 流水段标记：表示当该指令正在执行时，它在当前周期该指令所处的流水段。当模拟器工作在非流水方式下时，它没有意义。
- ◆ 符号指令：机器代码所对应的符号指令。

在该窗口中选中某行（用鼠标左键单击），然后再点击鼠标右键，就会弹出菜单：设置断点，清除断点，它们分别用于在所选指令处设置断点和清除断点。

- 设置断点

选择（点击）要设断点的指令→点击右键→“设置断点”，弹出“设置断点”小对话框，在“段”的下拉框中选择断点所在的流水段（在非流水方式下，不存在该下拉框），单击“确定”即可。

- 清除断点

选择（点击）指令→点击右键→“清除断点”，则设置在该指令处的断点被删除。

B.2.2 寄存器窗口

寄存器窗口显示 MIPSsim 模拟器中的寄存器的内容。共有 4 组寄存器：通用寄存器、浮点寄存器、特殊寄存器和流水寄存器，分为 4 栏来显示。每一栏下分别有各自的数据格式选项，如图 B.2 所示。

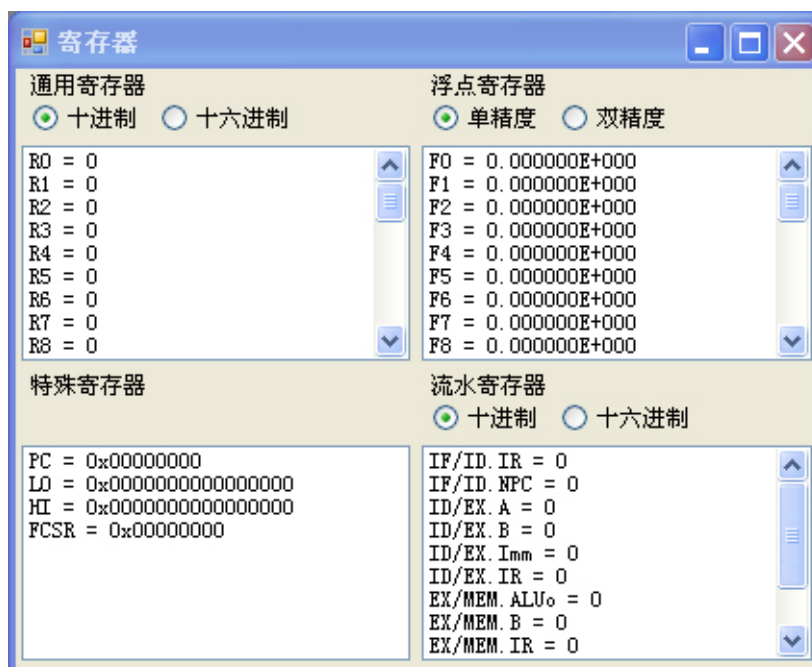


图 B.2 寄存器窗口

(1) 通用寄存器

MIPS64 有 32 个 64 位通用寄存器: R0, R1, ..., R31。它们被简称为 GPRs (General-Purpose Registers)，有时也被称为整数寄存器。R0 的值永远是 0。

通过数据格式选项，可以选择显示的格式是十进制还是十六进制。

(2) 浮点寄存器

共有 32 个 64 位浮点数寄存器: F0, F1, ..., F31。它们被简称为 FPRs (Floating-Point Registers)。它们既可以用来存放 32 个单精度浮点数 (32 位)，也可以用来存放 32 个双精度浮点数 (64 位)。存储单精度浮点数 (32 位) 时，只用到 FPR 的一半，其另一半没用。

(3) 特殊寄存器

特殊寄存器有 4 个：

- ◆ PC：程序计数器 (32 位)；
- ◆ LO：乘法寄存器的低位；
- ◆ HI：乘法寄存器的高位；
- ◆ FCSR：浮点状态寄存器。

(4) 流水寄存器

- ◆ IF/ID.IR：流水段 IF 与 ID 之间的指令寄存器；
- ◆ IF/ID.NPC：流水段 IF 与 ID 之间的下一指令程序计数器；
- ◆ ID/EX.A：流水段 ID 与 EX 之间的第一操作数寄存器；
- ◆ ID/EX.B：流水段 ID 与 EX 之间的第二操作数寄存器；

- ◆ ID/EX.Imm: 流水段 ID 与 EX 之间的立即数寄存器;
- ◆ ID/EX.IR: 存放从 IF/ID.IR 传过来的指令;
- ◆ EX/MEM.ALUo: 流水段 EX 与 MEM 之间的 ALU 计算结果寄存器;
- ◆ EX/MEM.IR: 存放从 ID/EX.IR 传过来的指令;
- ◆ MEM/WB.LMD: 流水段 MEM 与 WB 之间的数据寄存器, 用于存放从存储器读出的数据;
- ◆ MEM/WB.ALUo: 存放从 EX/MEM.ALUo 传过来的计算结果;
- ◆ MEM/WB.IR: 存放从 EX/MEM.IR 传过来的指令。

除了流水寄存器外, 其他寄存器都可以修改。只要双击某寄存器所在的行, 系统就会弹出一个对话框。该对话框显示了该寄存器原来的值。在新值框中填入新的值, 然后点击“保存”, 系统就会将新值写入该寄存器。

B.2.3 流水线窗口

流水线窗口显示流水线在当前配置下的组成以及该流水线的各段在当前周期正在处理的指令。如图 B.3 所示。

非流水方式下, 没有该窗口。

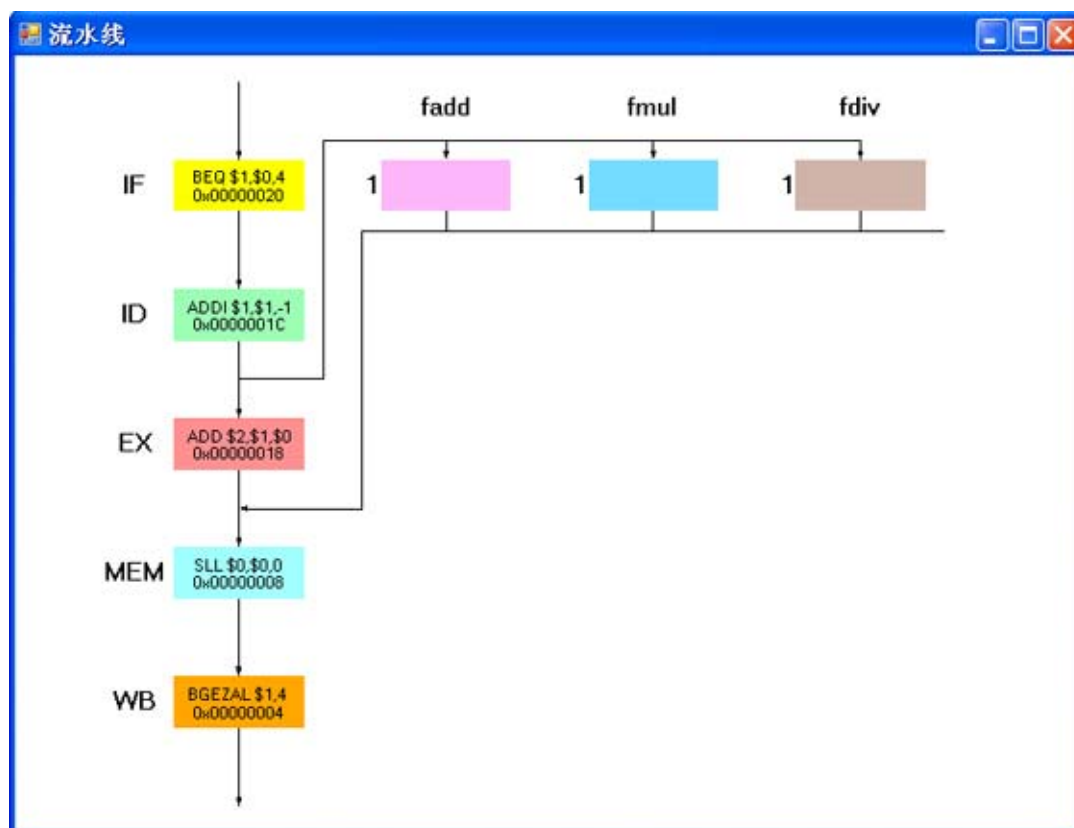


图 B.3 流水线窗口

在该窗口中, 每一个矩形方块代表一个流水段, 它们用不同的颜色填充。在该窗口的左侧是 IF 到 WB 段, 其右边为浮点部件。浮点部件分有浮点加法部件 (fadd)、浮点乘法部件 (fmul) 和浮点除法部件 (fdiv) 三种。在菜单“配置”→“常规配置”中修改浮点部件个数, 可看到该窗口中对应类型的浮点部件个数会发生相应的变化。

在运行过程中, 各段的矩形方块中会显示该段正在处理的指令及其地址 (16 进制)。当双击某矩形方块时, 会弹出窗口显示该段出口处的流水寄存器的内容 (16 进制)。

B. 2.4 时钟周期图窗口

该窗口用于显示程序执行的时间关系，画出各条指令执行时所用的时钟周期。非流水方式下，没有该窗口。

以窗口左上为原点，横轴正方向指向右方，表示模拟器先后经过的各个周期（列），纵轴正方向指向下方，表示模拟器中先后执行的各条指令（行）。如图 B.4 所示。

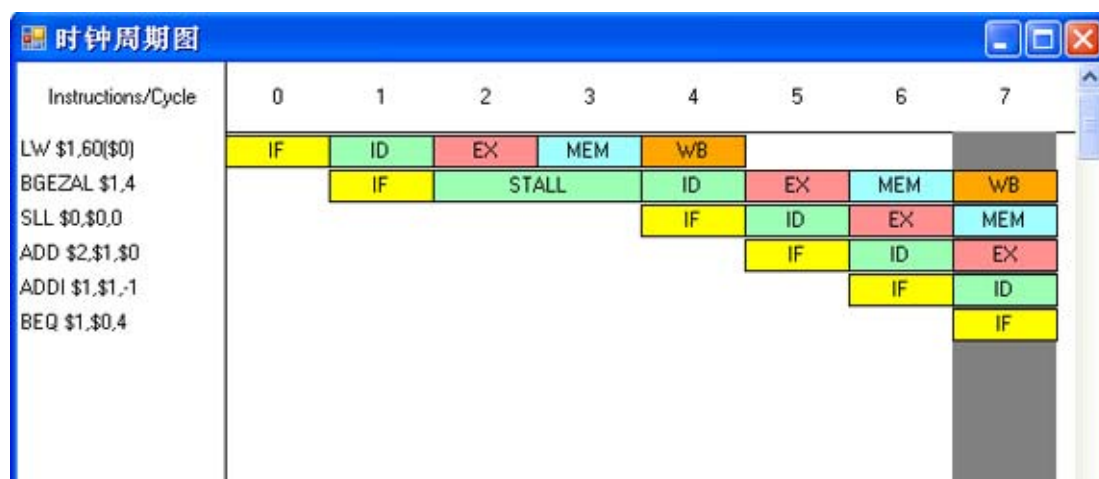


图 B.4 时钟周期图窗口

横坐标有相对周期计数和绝对周期计数两种不同的表示形式。在默认的绝对周期计数下，按 0、1、2、… 依次递增的顺序计数。而在相对周期计数下，当前周期记为第 0 个周期，而其余周期（在左边）则按其相对于当前周期的位置，分别记为-1，-2，-3，…等。

在由指令轴和周期轴组成的二维空间下，坐标 (n, i) 对应的矩形区域表示指令 i 在第 n+1 周期时所经过的流水段（假设采用绝对周期计数）。

双击某行时，会弹出一个小窗口，显示该指令在各流水段所进行的处理。

该窗口中还显示定向的情况。这是用箭头来表示的。若在第 m 周期和第 m+1 周期期间产生从指令 i1 到指令 i2 的定向，则在坐标 (m, i1) 和 (m+1, i2) 表示的矩形区域之间会有一个箭头。

B. 2.5 内存窗口

该窗口显示模拟器内存中的内容，左侧一栏为十六进制地址，右侧为数据，如图 B.5 所示。可以直接通过双击来修改其内容。这时会弹出一个“内存修改”对话框，如图 B.6 所示。对话框的上部区域为数据类型与格式选择区，通过勾选其中的一项，就可以指定所采用的数据类型与格式。

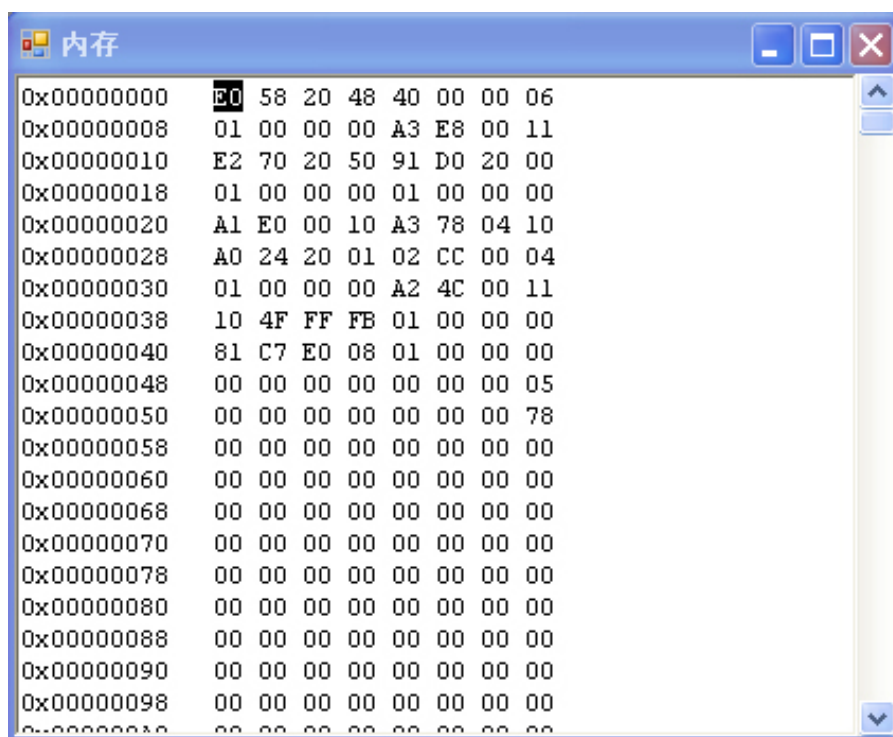


图 B.5 内存窗口

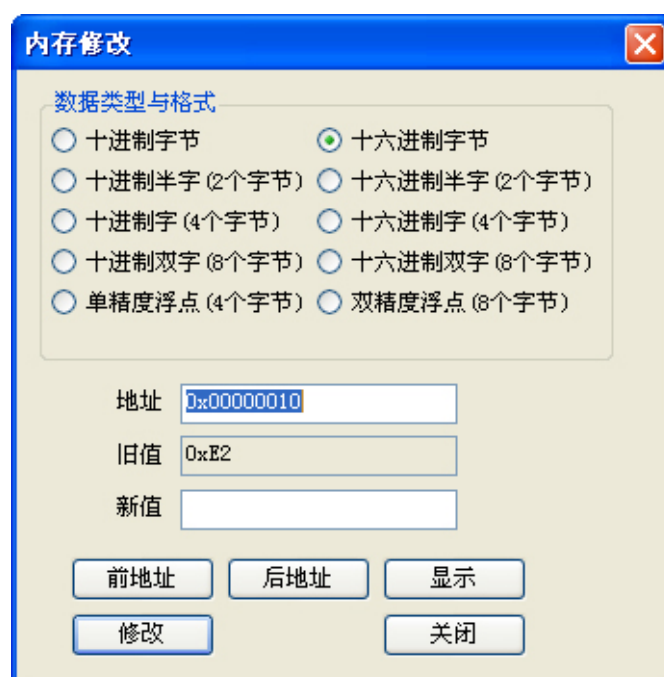


图 B.6 “内存修改”对话框

在该“内存修改”对话框中，地址框最开始显示的是被双击的单元的地址，用户可以直接修改该地址。在新值框中输入新值，然后点击按钮“修改”，模拟器就会把新值写入内存中相应的单元。新值的格式必须与所选的数据类型和格式一致。

“前地址”与“后地址”按钮分别将当前地址减少和增加一个数据长度（字节数），并

显示当前地址所指定单元的内容。“前地址”和“后地址”用于连续修改一片的内存数据。
“显示”按钮用于显示当前地址所指单元的内容。在修改地址后，点击该按钮就可以显示内存单元的内容。

B.2.6 统计窗口

该窗口显示模拟器统计的各项数据。如下所示。

(非流水方式下，没有该窗口)

汇总：

执行周期总数：0

ID 段执行了 0 条指令

硬件配置：

内存容量：4096 B

加法器个数：1 执行时间（周期数）：6

乘法器个数：1 执行时间（周期数）7

除法器个数：1 执行时间（周期数）10

定向机制：不采用

停顿（周期数）：

RAW 停顿：0 占周期总数的百分比：0%

其中：

load 停顿：0 占有所有 RAW 停顿的百分比：0%

分支/跳转停顿：0 占有所有 RAW 停顿的百分比：0%

浮点停顿：0 占有所有 RAW 停顿的百分比：0%

WAW 停顿：0 占周期总数的百分比：0%

结构停顿：0 占周期总数的百分比：0%

控制停顿：0 占周期总数的百分比：0%

自陷停顿：0 占周期总数的百分比：0%

停顿周期总数：0 占周期总数的百分比：0%

分支指令：

指令条数：0 占指令总数的百分比：0%

其中：

分支成功：0 占分支指令数的百分比：0%

分支失败：0 占分支指令数的百分比：0%

load/store 指令：

指令条数：0 占指令总数的百分比：0%

其中：

load：0 占 load/store 指令数的百分比：0%

store：0 占 load/store 指令数的百分比：0%

浮点指令：

指令条数: 0 占指令总数的百分比: 0%

其中:

加法: 0 占浮点指令数的百分比: 0%

乘法: 0 占浮点指令数的百分比: 0%

除法: 0 占浮点指令数的百分比: 0%

自陷指令:

指令条数: 0 占指令总数的百分比: 0%

B.2.7 断点窗口

断点一般是指指定的一条指令，当程序执行到该指令时，会中断执行，暂停在该指令上。在本模拟器中，断点可以设定在某条指令的某一个流水段上（如果是在流水方式下）。当该指令执行到相应的流水段时，会中断执行。

断点窗口列出当前已经设置的所有断点，每行一个。每行由 3 部分构成：地址（16 进制），流水段名称，符号指令。如图 B.7 所示。（在非流水方式下，“段”没有意义）



图 B.7 断点窗口

该窗口上方有四个按钮：添加、删除、全部删除、修改。

◆ 添加

单击“添加”，会弹出小对话框“设置断点”，在“地址”框中输入断点的十六进制地址，在“段”的下拉框中选择在哪个流水段中断（非流水方式下，不需要该操作，下同），单击“确定”即可。

◆ 删除

选中某个断点（单击断点列表中相应的一项），单击“删除”，则该断点被清除。

◆ 全部删除

单击“全部删除”，所有断点都将被清除。

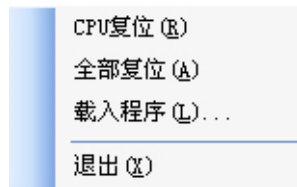
◆ 修改

选中某个断点，单击“修改”，会弹出小对话框“设置断点”，在“地址”框中输入断点的地址，在“段”的下拉框中选择在哪个流水段中断，单击“确定”即可将原断点修改为新设断点。

B.3 MIPSsim 的菜单

B.3.1 文件菜单

文件菜单如下所示：



(1) CPU 复位

将模拟器中 CPU 的状态复位为默认值。

(2) 全部复位

将整个模拟器的状态复位为默认值。模拟器启动时，也是将状态设置为默认值。

(3) 载入程序

将模拟器要执行的程序载入模拟器的内存。这个程序可以是汇编程序（.s 文件），也可以是汇编后的代码（.bin 文件）。点击该菜单后，系统将弹出“载入”对话框，选择要载入的文件，然后点击“打开”。如果是.s 文件，系统会对该文件进行汇编。若汇编过程无错误，则将产生的二进制代码载入至模拟器的内存；若有错误，则报告错误信息。如果是.bin 文件，则直接将该文件的内容载入到模拟器内存。

被载入程序在内存中连续存放，其起始地址默认为 0。该起始地址可以从“代码”菜单中的“载入起始地址”来查看和修改。修改时，如果输入的地址不是 4 的整数倍，模拟器会自动将其归整为 4 的整数倍。

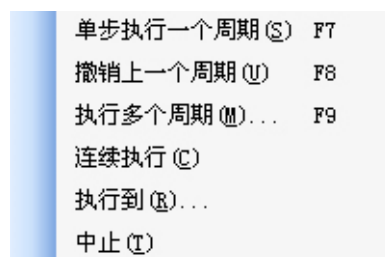
(4) 退出

退出模拟器。

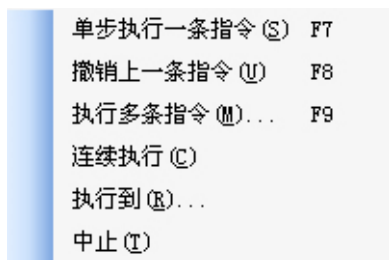
B.3.2 执行菜单

该菜单提供了对模拟器执行程序进行控制的功能。在下面的执行方式中，除了单步执行，当遇到断点或者用户手动中止（用“中止”菜单项）时，模拟器将立即暂停执行。

在流水方式下，执行菜单如下所示：



在非流水方式下，执行菜单如下所示：



(1) 单步执行一个周期

执行一个时钟周期，然后暂停。其快捷键为 F7。该菜单仅出现在流水方式下。

(2) 撤销上一个周期

模拟器回退一个时钟周期，即恢复到执行该周期之前的状态。其快捷键为 F8。该菜单仅出现在流水方式下。

(3) 执行多个周期

执行多个时钟周期，然后暂停。点击该菜单后，系统会弹出一个对话框，由用户指定要执行的周期的个数。该菜单仅出现在流水方式下。

(4) 连续执行

从当前状态开始连续执行程序，直到程序结束或遇到断点或用户手动中止。

(5) 执行到...

点击该菜单项后，系统会弹出一个“设定终点”对话框，由用户指定此次执行的终点，即在哪条指令的哪个流水段暂停。点击“确定”后，模拟器即开始执行程序，直到到达终点位置或遇到断点或用户手动中止。所输入的终点地址将被归整为 4 的整数倍。

(6) 中止

点击该菜单项后，模拟器会立即暂停执行。当模拟器执行程序出现长期不结束的状况时，可用该菜单强行使模拟器停止执行。

(7) 单步执行一条指令

执行一条指令，然后暂停。该指令的地址由当前的 PC 给出。其快捷键为 F7。该菜单仅出现在非流水方式下。

(8) 撤销上一条指令

模拟器回退一条指令，即恢复到执行该指令之前的状态。其快捷键为 F8。该菜单仅出现在非流水方式下。

(9) 执行多条指令

执行多条指令，然后暂停。点击该菜单项后，系统会弹出一个对话框，由用户指定要执行的指令的条数。

B.3.3 内存菜单

该菜单下有 3 项：显示，修改，符号表。

(1) 显示

该菜单项用于设置显示内存的值的数据类型与格式。点击该菜单项后，系统会弹出“内存显示”对话框，如图 B.8 所示。在选定所要的数据类型与格式后，单击“确定”按钮，内存窗口中的数据就会按指定的数据类型与格式显示。

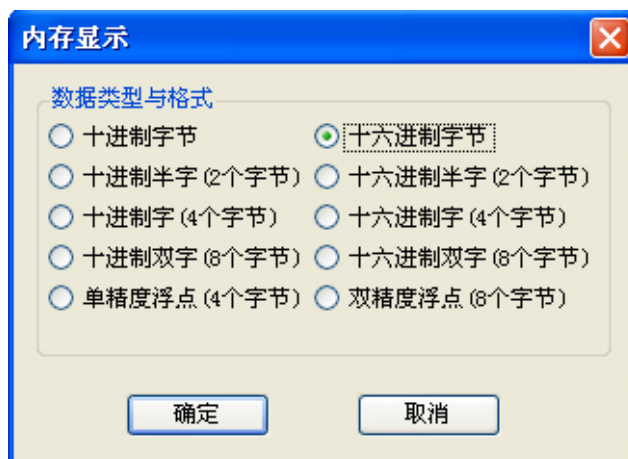


图 B.8 内存显示窗口

(2) 修改

该菜单项用于对内存的值进行修改。点击该菜单项后，系统会弹出“内存修改”对话框，该对话框与图 6 的“内存修改”相同。请参见相关的论述。

(3) 符号表

该菜单项用于显示符号表。符号表中列出了当前被执行的程序中各符号的地址（内存地址）。

B. 3.4 代码菜单

(1) 载入起始地址

该菜单项用于显示和修改代码载入内存的起始地址。如果要修改，只要在弹出的“载入起始地址”对话框中输入新的地址，然后点击“确认”即可。

(2) 设置断点

点击该菜单项，会弹出“设置断点”小对话框，输入断点地址，并在“段”的下拉框中选择断点所在的流水段（在非流水方式下，不存在该下拉框），单击“确定”即可。

(3) 取消断点

在代码窗口中选择某条指令，然后点击该菜单项，则设置在该指令处的断点被删除。

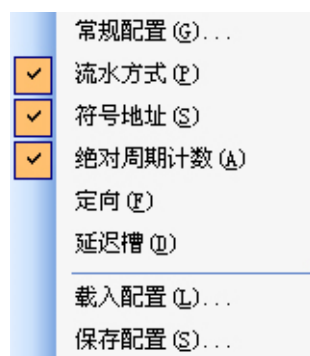
(4) 清除所有断点

清除所有断点。

B. 3.5 配置菜单

配置菜单用于修改模拟器的配置。需要注意的是：修改配置将会使模拟器复位。

配置菜单如下所示：



(1) 常规配置

该菜单项用于查看和设置以下参数：（其默认值见“一、启动程序”）

- ◆ 内存容量；
- ◆ 浮点加法运算部件个数
- ◆ 浮点乘法运算部件个数
- ◆ 浮点除法运算部件个数
- ◆ 浮点加法运算延迟周期数
- ◆ 浮点乘法运算延迟周期数
- ◆ 浮点除法运算延迟周期数

如果要修改这些参数，只要直接修改，然后点击“确定”按钮即可。

(2) 流水方式

“流水方式”开关。当该项被勾选（即其前面有个“√”号）时，模拟器按流水方式工作，能模拟流水线的工作过程。否则（即没有被勾选）就是按串行方式执行程序。

该项的勾选和去选（即去掉其前面的“√”号）都是通过点击该项来实现的。下同。

当从流水方式切换为非流水方式（或相反）时，系统将强行使模拟器的状态复位。

(3) 符号地址

“符号地址”开关。当该项被勾选时，代码窗口中的代码将显示原汇编程序中所采用的标号。否则（即没有被勾选）就不显示。

(4) 绝对周期计数

“绝对周期计数”开关。当该项被勾选时，时钟周期图窗口中的横坐标将显示为绝对周期，即时钟周期按 0、1、2、… 顺序递增的顺序计数。否则（即没有被勾选）就按相对周期计数，即把当前周期记为第 0 个周期，而其余周期（在左边）则按其相对于当前周期的位置，分别记为-1, -2, -3, …等。

在非流水方式下，该菜单项不起作用（变灰）。

(5) 定向

“定向”开关，用于指定是否采用定向技术。当该项被勾选时，模拟器在执行程序时将采用定向技术。否则就不采用。

在非流水方式下，该菜单项不起作用（变灰）。

(6) 延迟槽

“延迟槽”开关，用于指定是否采用延迟分支技术。当该项被勾选时，模拟器在执行程序时将采用一个延迟槽。否则就不采用。

在非流水方式下，该菜单项不起作用（变灰）。

(7) 载入配置

用于将一个配置文件(.cfg)载入模拟器，模拟器将按该文件进行配置并复位。点击该菜单项，系统将弹出“打开”对话框，让用户指定所要载入的配置文件。

(8) 保存配置

用于将模拟器当前的配置保存到一个配置文件(.cfg)中，以便以后重用。点击该菜单项，系统将弹出“保存文件”对话框，让用户指定配置文件的名称和位置。

B.3.6 窗口菜单

(1) 平铺

将当前已打开的子窗口平铺在主窗口中。

(2) 层叠

将当前已打开的子窗口层叠在主窗口中。

(3) 打开所有

将所有子窗口都打开。

(4) 收起所有

将所有子窗口最小化。

(5) 选择子窗口

在该菜单中还列出了所有的子窗口的名称，选择其中的任一个，将使该窗口被选中，并被提到最上层。

B. 3. 7 帮助菜单

该菜单下有两项：“帮助”和“关于 MIPSsim”。前者提供用户手册，后者给出关于 MIPSsim 的版权信息和设计开发者信息。

附录 C MIPS16 模拟器及相关指令

C.1 MIPS16 模拟器的存储单元

1. 寄存器

共有 3 组寄存器：通用寄存器、浮点寄存器、特殊寄存器。

(1) 通用寄存器

此模拟器有 32 个 64 位通用寄存器：R0, R1, ..., R31。它们被简称为 GPRs (General-Purpose Registers)，有时也被称为整数寄存器。R0 的值永远是 0。

在 MIPS16 中，整数用补码表示。

(2) 浮点寄存器

共有 32 个 64 位浮点数寄存器：F0, F1, ..., F31。它们被简称为 FPRs (Floating-Point Registers)。它们既可以用来存放 32 个单精度浮点数 (32 位)，也可以用来存放 32 个双精度浮点数 (64 位)。存储单精度浮点数时，只用到 FPR 的一半，其另一半没用。

在 MIPS16 中，浮点数采用 IEEE754 标准来表示。其格式如下：

单精度浮点数共 32 位，由 1 位符号位、8 位阶码、23 位尾数组成。

- 符号位：浮点数的符号，0 表示正数，1 表示负数。
- 阶码：浮点数指数域的编码值，用移码表示。
- 尾数：浮点数的二进制小数，用原码表示。

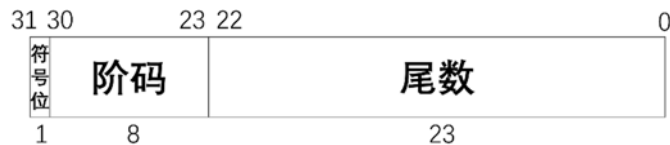


图 1.1 单精度浮点数

双精度浮点数共 64 位，由 1 位符号位、11 位阶码、52 位尾数组成。



图 1.2 双精度浮点数

(3) 特殊寄存器

特殊寄存器有 3 个：

PC：程序计数器 (32 位)。

L0：乘法结果寄存器的低位 (64 位)。

HI：乘法结果寄存器的高位 (64 位)。

模拟器启动时，所有这些寄存器都被初始化为 0。

2. 存储器

存储器的大小为 512 个字，共有 2048 个字节，按字寻址。程序载入时，从第 0 号单元开始存放。

C.2 MIPS16 模拟器的功能

1. 载入程序

命令格式: load filename.s

功能说明: 将模拟器要执行的程序 filename.s 载入模拟器的内存。从第 0 号单元开始连续存放，每条指令占 4 个字节。

2. 单步执行一条指令

命令格式: step

功能说明: 执行一条指令，然后暂停。所执行指令的地址由当前的 PC 值给出。执行完后，PC 值+4（因为每条指令占 4 个字节）。

3. 连续执行到底

命令格式: srun

功能说明: 从当前的 PC 值的位置开始连续执行程序，直至结束。

4. 显示

将寄存器或存储器的内容显示到屏幕上。默认是按 16 进制显示。

（1）显示通用寄存器内容

命令格式: disp r[/10]

功能说明: 显示通用寄存器的内容，默认按 16 进制显示。下同。[/10]为可选项。

- /10 按 10 进制显示。

（2）显示浮点寄存器内容

命令格式: disp f[/f]/[df]

功能说明: 显示浮点寄存器的内容。[/f]和[/df]为可选项，最多选一个。下同。

- /f 按 32 位浮点数显示。
- /df 按 64 位浮点数显示。

（3）显示特殊寄存器内容

命令格式: disp s[/10]

功能说明: 显示特殊寄存器的内容。

- /10 按 10 进制显示。

（4）显示存储器内容

命令格式: disp mem[/p]/[i]/[di]/[f]/[df] [addr]

功能说明: 从存储器中 addr（默认为 0x00000000）开始显示内容，默认显示 32 个数据。

[/p]/[i]/[di]/[f]/[df]为可选项，最多选一个。

- /p 按指令显示。
- /i 按 32 位整数显示。
- /di 按 64 位整数显示。

- /f 按 32 位浮点数显示。
 - /df 按 64 位浮点数显示。
- addr 的值必须是 4 的整数倍。

5. 转储

命令格式：dump filename.txt

功能说明：将系统的状态信息（寄存器和存储器的内容）保存到文件 filename.txt。

6. 复位

命令格式：rset

功能说明：将整个模拟器的状态复位为默认值（把寄存器和存储器单元置为 0）。模拟器启动时，也是将状态设置为默认值。

7. 帮助

命令格式：help

功能说明：显示模拟器的所有可执行操作。

8. 退出

命令格式：exit

功能说明：退出 MIPS16 模拟器。

C. 3 MIPS16 模拟器的部分变量说明

为了便于实验者编写指令解释函数 execute()函数，下面对需要用到的变量进行说明。

1. currentInst

currentInst 中存放当前指令，即程序计数器 PC 当前指向的指令。此变量的字段按照 R 型指令、浮点（F）型指令、立即数（I）型指令的格式分别进行命名。例如对于 DADD 指令，其为 R 型指令，要访问其 rs 字段，用 currentInst.R.rs 即可。

currentInst 使用 Inst 结构体，如下所示。

```
typedef struct{
    uint8_t opcode:6;
    union{
        struct{
            uint8_t rs:5;
            uint8_t rt:5;
            uint8_t rd:5;
            uint8_t shamt:5;
            uint8_t func:6;
        } R;
        struct{
```

```

        uint8_t fmt:5;
        uint8_t ft:5;
        uint8_t fs:5;
        uint8_t fd:5;
        uint8_t func:6;
    } F;
    struct{
        uint8_t rs:5;
        uint8_t rt:5;
        uint32_t imme:16;
    } I;
};
} Inst;

```

2. state

state 存储模拟器中的寄存器信息，其具体信息如下。

- regs_r 通用寄存器
- regs_f 浮点寄存器
- PC 程序计数器
- LO 乘法结果寄存器的低位
- HI 乘法结果寄存器的高位

例如要访问通用寄存器中 R1 的值，采用 state.regs_r[1]即可。其余类似。

state 使用 State 结构体，如下所示。

```

typedef struct{
    uint64_t regs_r[REGSIZE]; //通用寄存器
    double regs_f[REGSIZE]; //浮点寄存器
    uint32_t PC; //程序计数器
    uint64_t LO; //乘法结果寄存器的低位
    uint64_t HI; //乘法结果寄存器的高位
} State;

```

3. memory

数组 memeory[512]模拟存储器，按字寻址。每个字由 4 个字节组成，但在交互命令和显示内容中，使用字节地址，它等于字地址乘以 4。

C. 4 MIPS16 模拟器支持的伪指令

MIPS16 模拟器支持.text、.data、.word、.double 四种伪指令，下面对这四种伪指令进行说明。

1. .text

格式: .text

说明: 定义程序的代码段, 该指令后面就是程序代码。该语句单独占一行。

2. .data

格式: .data

说明: 定义程序的数据段, 必须放在.text 和程序代码之后。该语句单独占一行。

3. .word

格式: name: .word value

说明: name 为变量名, 其后必须跟冒号。value 为十进制整数, 只支持一个变量值, 不支持数组。该语句单独占一行。

4. .double

格式: name: .double value

说明: name 为变量名, 其后必须跟冒号。value 为十进制双精度浮点数, 只支持一个变量值, 不支持数组。该语句单独占一行。

C.5 模拟指令集合

此模拟器能够模拟执行 16 条 MIPS 指令, 下面详细说明之。

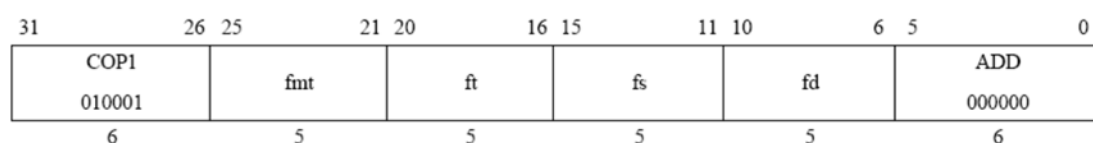
符号说明:

- (1) 在指令助记符中, “W”表示 32 位整数, “L”表示 64 位整数, “S”表示单精度浮点数, “D”表示双精度浮点数。“fmt”表示多种格式的数据, $fmt \in (S, D, W, L)$ 。
- (2) 助记符的最后一个字母为 U 表示无符号操作, I 表示与立即值操作, IU 表示无符号立即值操作。助记符的第一个字母为 D 表示是双字 (64 位) 操作。
- (3) 为简洁起见, 直接用 rs 来表示 rs 寄存器中的内容, 其他的如 rt, rd, fs, ft, fd 等都是如此。
- (4) fs, ft, fd 表示浮点寄存器。一般来说, fs 和 ft 表示源操作数 (寄存器), fd 表示结果寄存器。
- (5) rs, rt, rd 表示整数寄存器, 也称为通用寄存器。一般来说, rs 和 rt 表示源操作数 (寄存器), rd 表示目的寄存器。
- (6) 以下是两个特殊寄存器:
LO——特殊寄存器, 常用来存放乘积的低 64 位;
HI——特殊寄存器, 常用来存放乘积的高 64 位。

1. ADD.D

名称: 双精度浮点加

指令格式:



符号指令:

ADD.D fd, fs, ft

在MIPS16汇编程序中，寄存器号前都加了\$符，详见sample.s。下同。

功能说明：

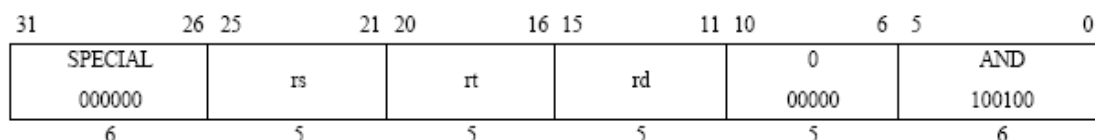
$fd \leftarrow fs + ft$

对浮点寄存器fs和ft中的双精度浮点数进行加法运算，结果放入浮点寄存器fd。

2. AND

名称：与

指令格式：



符号指令：

AND rd, rs, rt

功能说明：

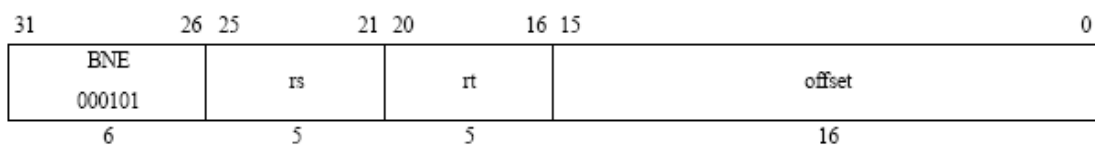
$rd \leftarrow rs \text{ AND } rt$

把rt和rs中的数据按位进行逻辑“与”操作，结果放入rd。

3. BNE

名称：不相等转移

指令格式：



符号指令：

BNE rs, rt, offset

功能说明：

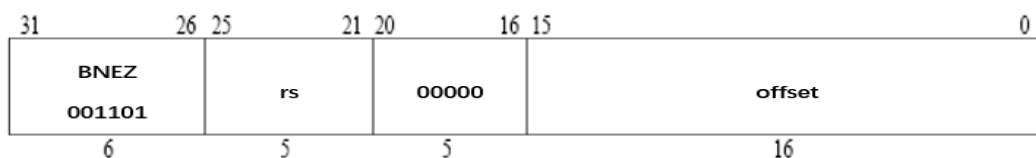
if(rs!=rt) then 转移

转移地址：将16位的offset左移2位并进行符号位扩展后，与PC相加。

4. BNEZ

名称：不等于0转移

指令格式：



符号指令：

BNEZ rs, offset

功能说明：

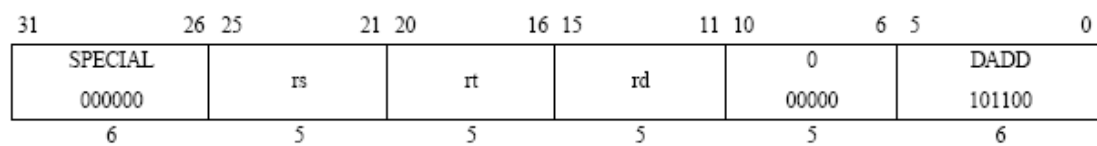
if(rs!=0) then 转移

转移地址：将16位的offset左移2位并进行符号位扩展后，与PC相加。

5. DADD

名称：双字寄存器加

指令格式：



符号指令：

DADD rd, rs, rt

功能说明：

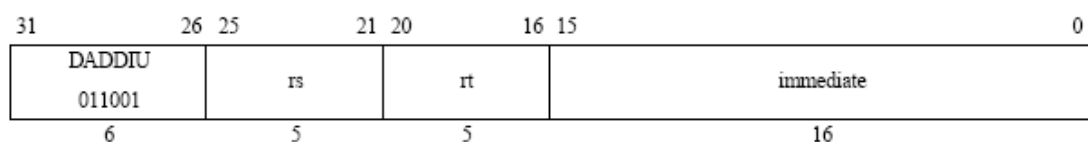
$rd \leftarrow rs + rt$

对rs和rt中的64位的整数进行加法运算，结果放入rd。按有符号数操作。

6. DADDIU

名称：双字无符号立即值加

指令格式：



符号指令：

DADDIU rt, rs, immediate

功能说明：

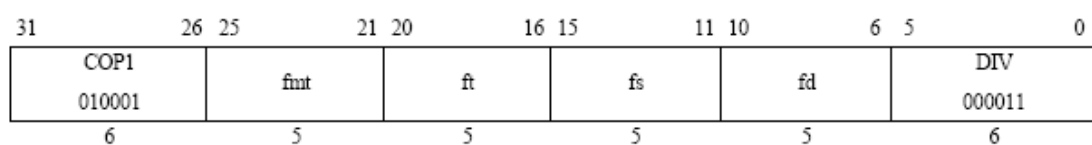
$rt \leftarrow rs + \text{immediate}$

把带符号的16位立即值immediate与rs中的64位整数相加，结果放入rt。按无符号数操作。

7. DIV.D

名称：双精度浮点除

指令格式：



符号指令：

DIV.D fd, fs, ft

功能说明：

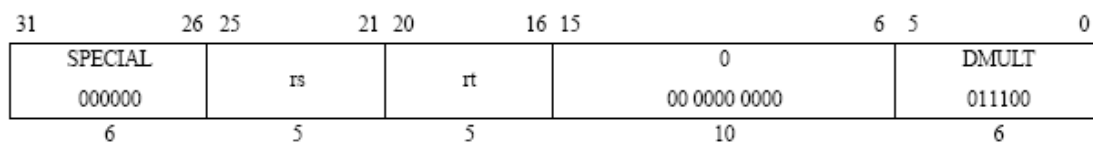
$fd \leftarrow fs / ft$

把两个浮点寄存器fs和ft中的双精度浮点数进行除法运算，结果放入fd。

8. DMULT

名称：双字寄存器乘

指令格式：



符号指令：

DMULT rs, rt

功能说明：

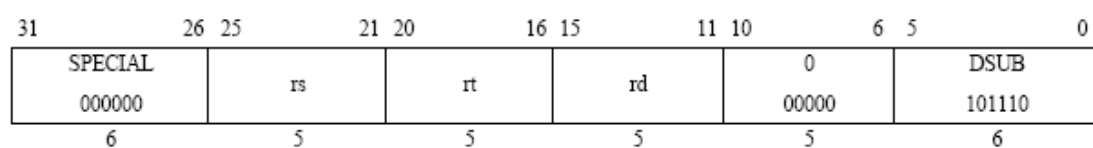
$(LO, HI) \leftarrow rs \times rt$

两个64位整数相乘，产生128位的积。其低64位送LO，高64位送HI。按有符号数操作。

9. DSUB

名称：双字寄存器减

指令格式：



符号指令：

DSUB rd, rs, rt

功能说明：

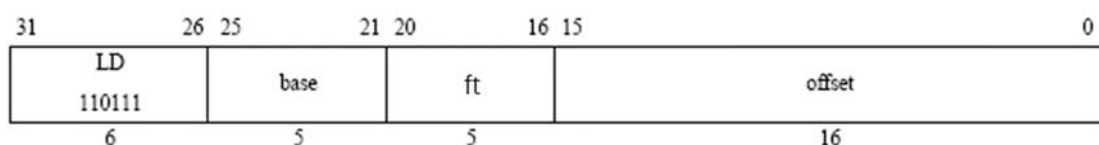
$rd \leftarrow rs - rt$

整数寄存器rs和rt中的两个64位整数进行减法运算，结果送入rd。按有符号数操作。

10. L.D

名称：取双精度浮点数

指令格式：



符号指令：

L.D ft, offset(base)

功能说明：

$ft \leftarrow \text{memory}[\text{base} + \text{offset}]$

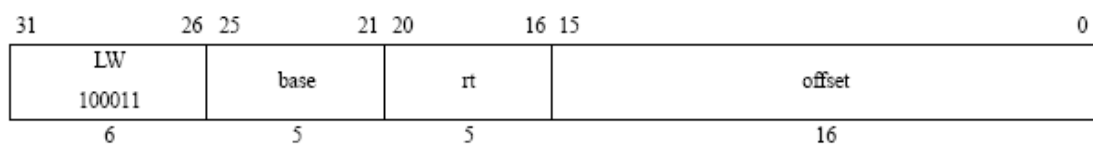
从存储器读出一个 64 位的数据，送入 ft。

访存地址：寄存器base的内容+经有符号32位扩展后的偏移量offset。

11. LW

名称：取字

指令格式：



符号指令：

LW rt, offset(base)

功能说明：

$rt \leftarrow \text{memory}[\text{base} + \text{offset}]$

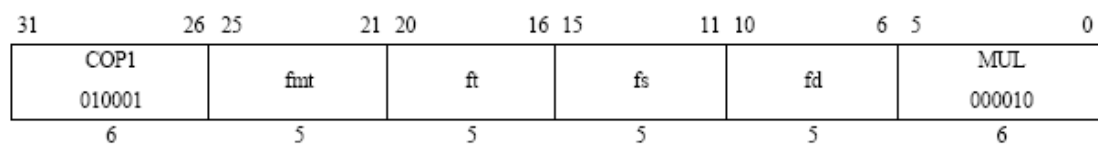
从存储器读出一个 32 位数据，按符号位扩展后，送入 rt。

访存地址：寄存器base的内容+经有符号32位扩展后的偏移量offset。

12. MUL.D

名称：双精度浮点乘

指令格式：



符号指令：

MUL.D fd, fs, ft

功能说明：

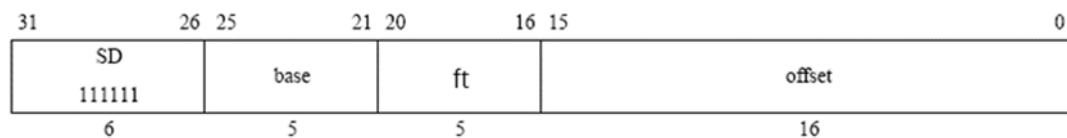
$fd \leftarrow fs \times ft$

浮点寄存器 fs 和 ft 中的双精度浮点数进行乘法运算，结果放入 fd。

13. S.D

名称：存双精度浮点数

指令格式：



符号指令：

S.D ft, offset(base)

功能说明：

$\text{memory}[\text{base} + \text{offset}] \leftarrow ft$

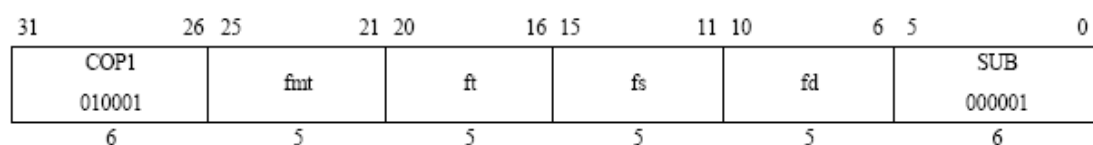
将通用寄存器 rt 中的双字写入存储器。

访存地址：寄存器base的内容+有符号32位扩展后的偏移量offset。

14. SUB.D

名称：双精度浮点减

指令格式：



符号指令：

SUB.D fd, fs, ft

功能说明：

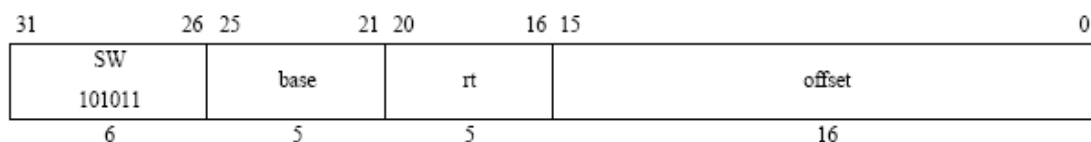
$$fd \leftarrow fs - ft$$

寄存器 fs 和寄存器 ft 中的双精度浮点数进行减法运算，结果送入浮点寄存器 fd。

15. SW

名称：存字

指令格式：



符号指令：

SW rt, offset(base)

功能说明：

$\text{memory}[\text{base} + \text{offset}] \leftarrow \text{rt}$

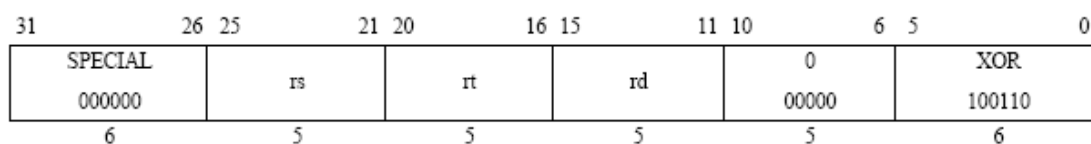
将 rt 中的低 32 位数据写入存储器。

访存地址：寄存器 base 的内容 + 有符号 32 位扩展后的偏移量 offset。

16. XOR

名称：异或

指令格式：



符号指令：

XOR rd, rs, rt

功能说明：

$rd \leftarrow rs \text{ XOR } rt$

通用寄存器 rs 和 rt 中的两个 32 位数按位进行逻辑“异或”操作，结果放入 rd。

C. 6 样例程序 sample.s

说明：在汇编程序中，在寄存器号前，都加上了 \$ 符。

```
.text
main:
DADDIU  $r2,$r1,A
DADD    $r3,$r1,$r2
AND     $r5,$r2,$r3
BNE     $r1,$r2,loop
DADD    $r7,$r2,$r5
loop:
SW      $r2,0($r2)
```

```
LW    $r1,0($r2)
DADDIU $r4,$r8,4
DSUB   $r6,$r2,$r4
L.D    $f1,80($r0)
BNEZ   $r2,branch
DADD    $r9,$r2,$r5
branch:
ADD.D   $f3,$f1,$f2
DIV.D   $f4,$f3,$f1
SUB.D   $f5,$f3,$f4
XOR     $r8,$r1,$r6
DMULT   $r2,$r2
MUL.D   $f6,$f1,$f5
S.D     $f5,16($r2)
.data
A: .word 128
B: .double 256.5
```